University of Sheffield

COM3529 Software Testing and Analysis

White-Box Coverage Criteria based on Control Flow Analysis

Dr José Miguel Rojas

Control Flow

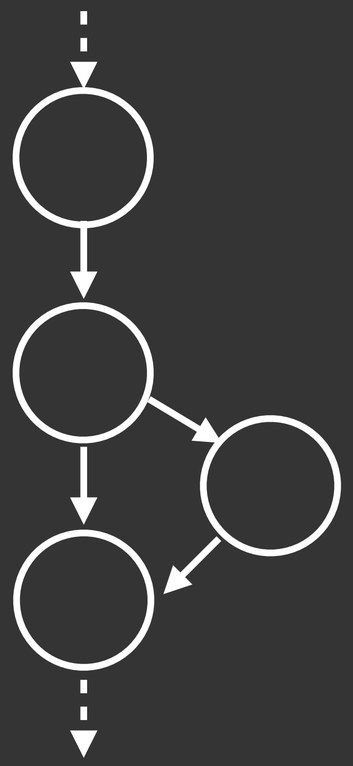
```java
public static Set<Character> duplicateLetters(String s) {
    // lower case the string and remove all characters
    // that are not letters
    s = s.toLowerCase().replaceAll("[^a-z.]", "");

    // initialise the result set
    Set<Character> duplicates = new TreeSet<>();

    // iterate through the string
    for (int i = 0; i < s.length(); i++) {
        char si = s.charAt(i);

        // iterate through the rest of the string,
        // checking for the same letter
        for (int j = i + 1; j < s.length(); j++) {
            char sj = s.charAt(j);

            if (si == sj) {
                // a match has been found, add it to
                // the result set
                duplicates.add(si);
            }
        }
    }

    return duplicates;
}
```

Control Flow Graph

```java
public static Set<Character> duplicateLetters(String s) {
    // lower case the string and remove all characters
    // that are not letters
    s = s.toLowerCase().replaceAll("[^a-z.]", "");

    // initialise the result set
    Set<Character> duplicates = new TreeSet<>();

    // iterate through the string
    for (int i = 0; i < s.length(); i++) {
        char si = s.charAt(i);

        // iterate through the rest of the string,
        // checking for the same letter
        for (int j = i + 1; j < s.length(); j++) {
            char sj = s.charAt(j);

            if (si == sj) {
                // a match has been found, add it to
                // the result set
                duplicates.add(si);
            }
        }
    }

    return duplicates;
}
```

Control Flow Graph

(CFG)



```java
public static Set<Character> duplicateLetters(String s) {
    // lower case the string and remove all characters
    // that are not letters
    s = s.toLowerCase().replaceAll("[^a-z.]", "");

    // initialise the result set
    Set<Character> duplicates = new TreeSet<>();

    // iterate through the string
    for (int i = 0; i < s.length(); i++) {
        char si = s.charAt(i);

        // iterate through the rest of the string,
        // checking for the same letter
        for (int j = i + 1; j < s.length(); j++) {
            char sj = s.charAt(j);

            if (si == sj) {
                // a match has been found, add it to
                // the result set
                duplicates.add(si);
            }
        }
    }

    return duplicates;
}
```

Linear Sequences of Statements

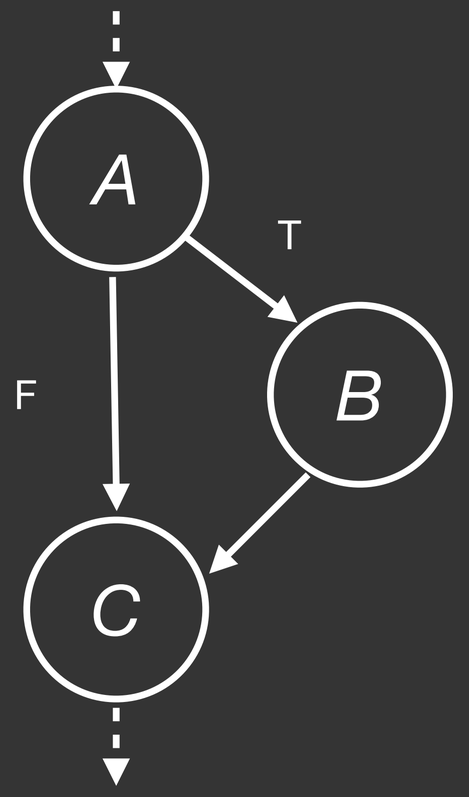
```
doSomething(); /* A */
doSomethingElse(); /* B */
doSomethingDifferent(); /* C */
```

Linear Sequences of Statements

```
doSomething(); /* A */
doSomethingElse(); /* B */
doSomethingDifferent(); /* C */
```

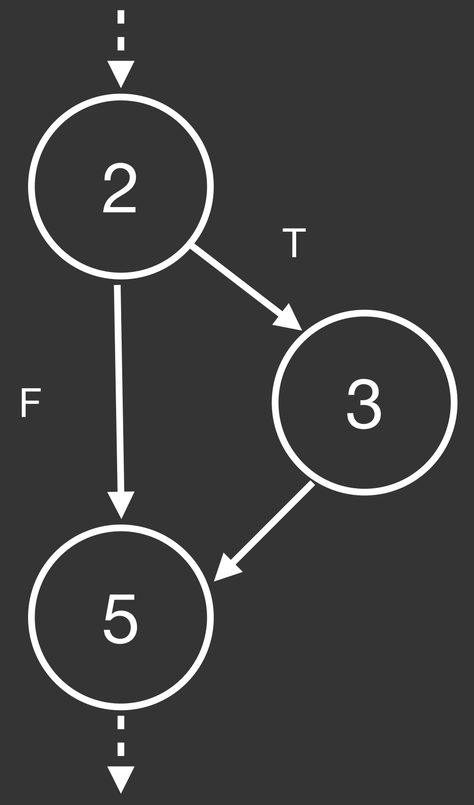
“If” Constructs

```
if (condition) { /*A*/
  doSomething(); /* B */
}
/* C */
```



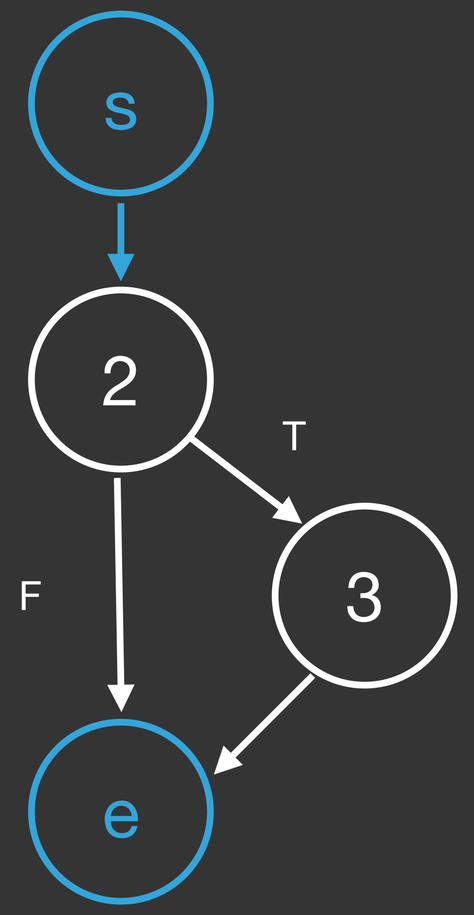
Methods

```
1 public void printGreeting() {
2   if (isMorning()) {
3     System.out.println("Good Morning!");
4   }
5 }
```



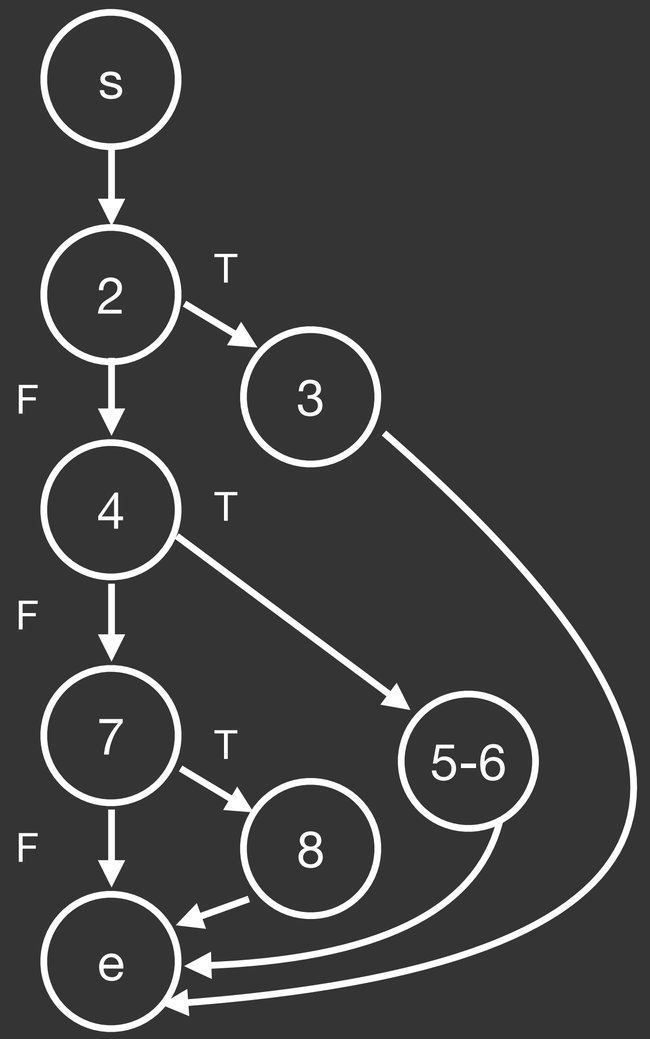
Methods

```
1 public void printGreeting() {
2   if (isMorning()) {
3     System.out.println("Good Morning!");
4   }
5 }
```



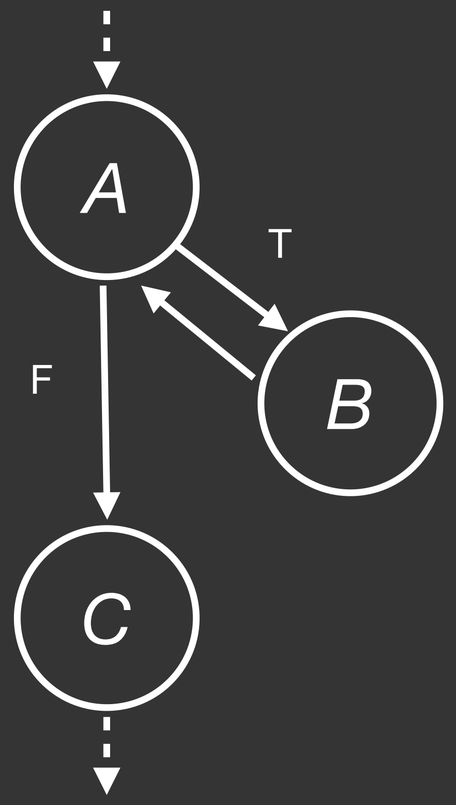
Nesting

```
1 public void printGreeting() {
2   if (isMorning()) {
3     System.out.println("Good Morning!");
4   } else if (isAfternoon()) {
5     System.out.println("Good Afternoon!");
6     System.out.println("Lovely day for a stroll!");
7   } else if (isEvening()) {
8     System.out.println("Good Evening!");
9   }
10 }
```



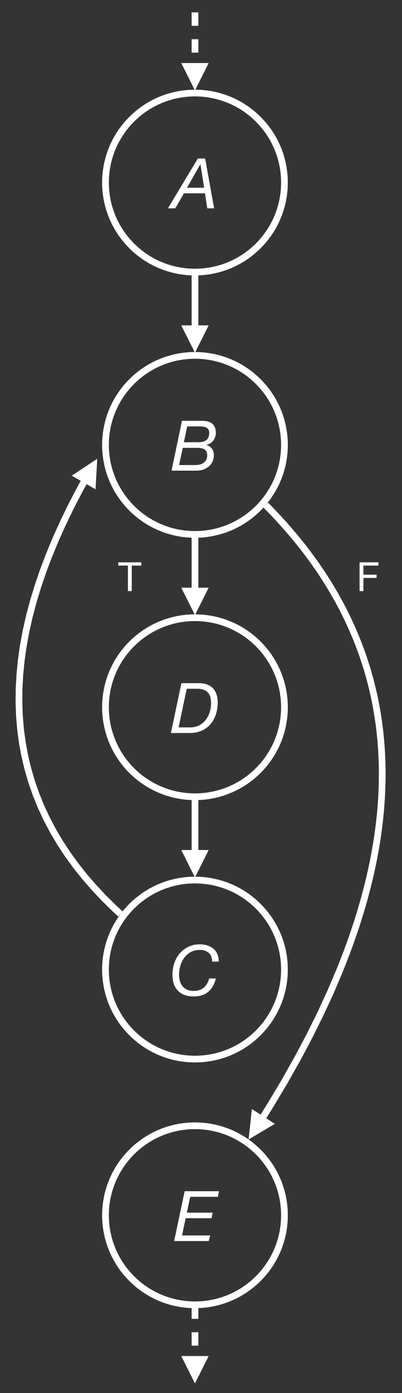
“While” Constructs

```
while (condition) { /*A*/
  doSomething(); /* B */
}
/* C */
```

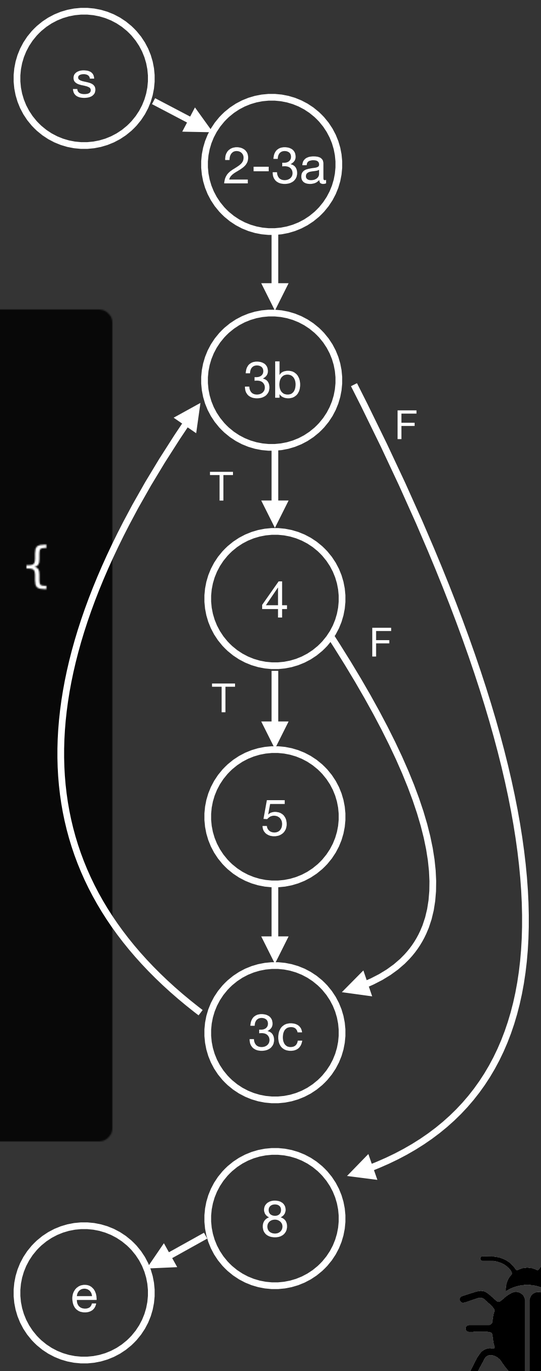


“For” Constructs

```
for (/*A*/ int i = 0; /*B*/ i < 10; /*C*/ i ++) {
  doSomething(); /* D */
}
// E
```



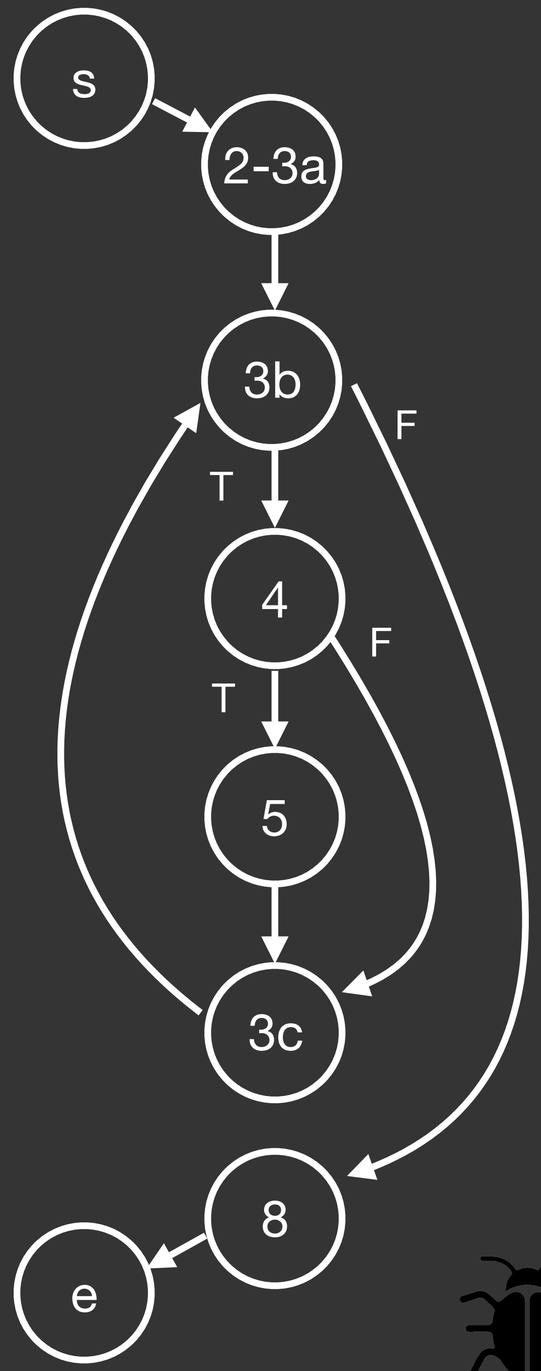
```java
public int countZeros(int[] x) {
  int count = 0;
  for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
    if (x[i] == 0) {
        count ++;
    }
  }
  return count;
}
```



Statement Coverage

The test suite should execute all nodes of the CFG

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Statement Coverage

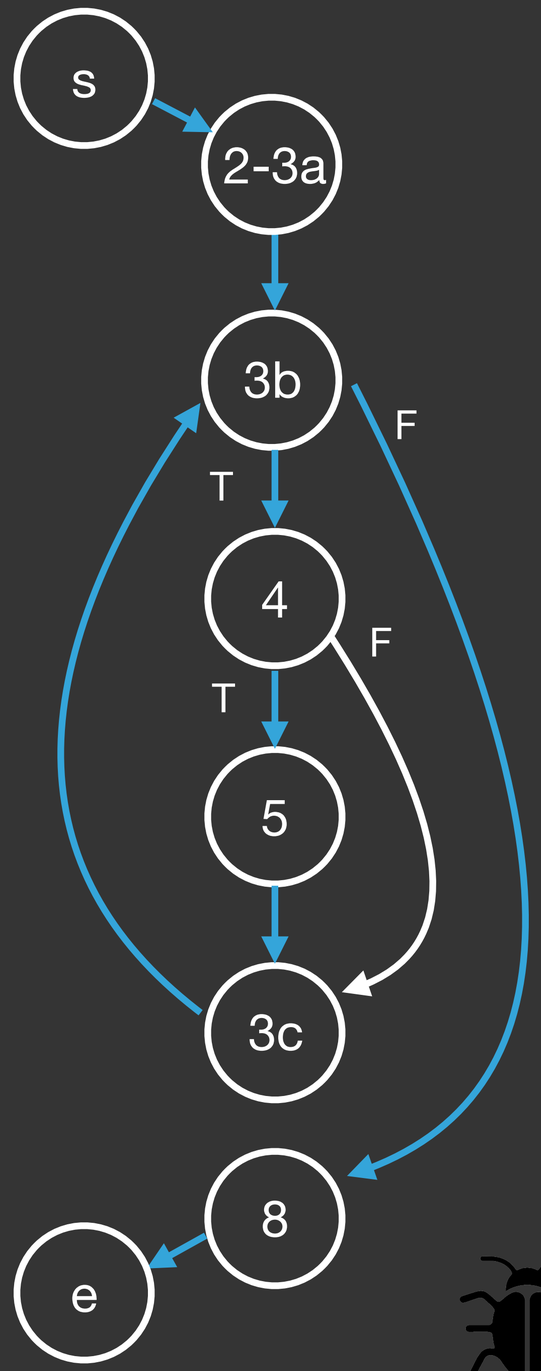
The test suite should execute all nodes of the CFG

The test case x = [0] would execute all nodes

It takes the path s → 2-3a → 3b → 4 → 5 → 3c → 3b → e

(In practice covering all nodes may need several test cases)

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Statement Coverage

The test suite should execute all nodes of the CFG

Also known as:

- Line Coverage
- Node Coverage
- Basic Block Coverage

Statement Coverage

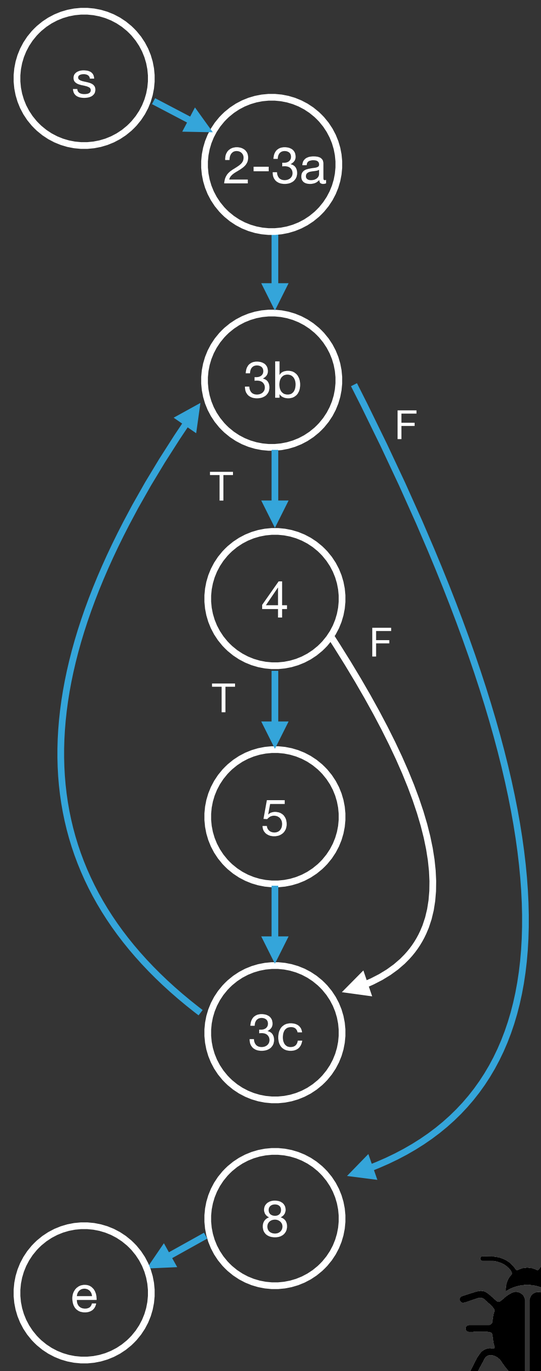
The test suite should execute all nodes of the CFG

The test case x = [0] would execute all nodes

It takes the path s → 2-3a → 3b → 4 → 5 → 3c → 3b → e

(In practice covering all nodes may need several test cases)

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Statement Coverage

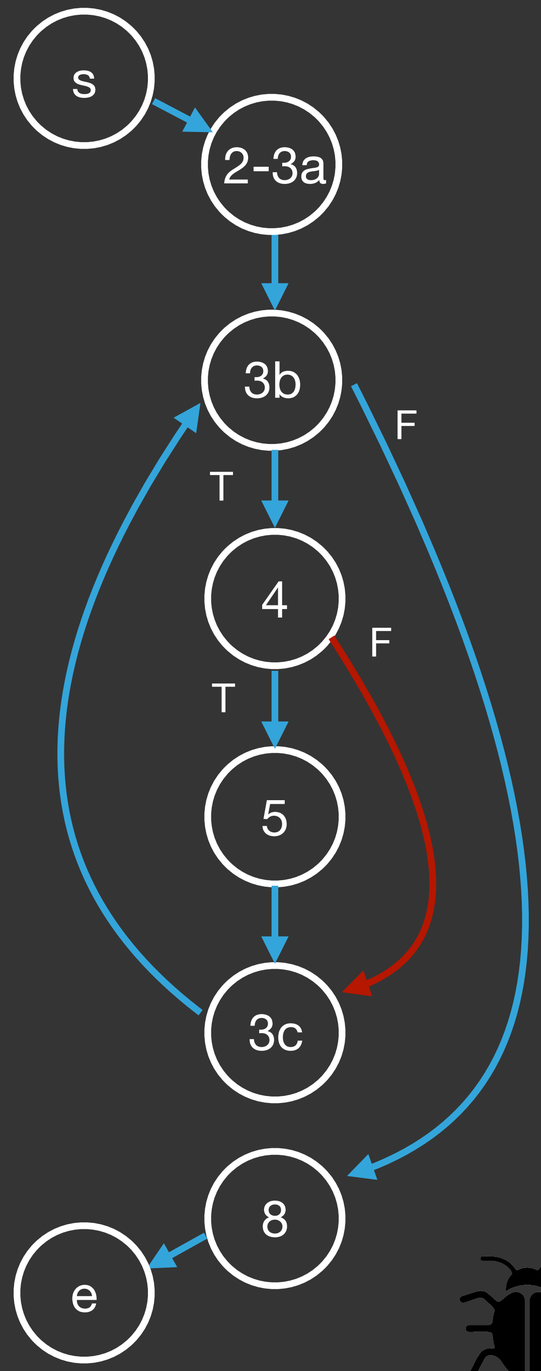
The test suite should execute all nodes of the CFG

The test case x = [0] would execute all nodes

It takes the path s → 2-3a → 3b → 4 → 5 → 3c → 3b → e

(In practice covering all nodes may need several test cases)

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Branch Coverage

The test suite should execute all true/false edges of the CFG

Branch Coverage

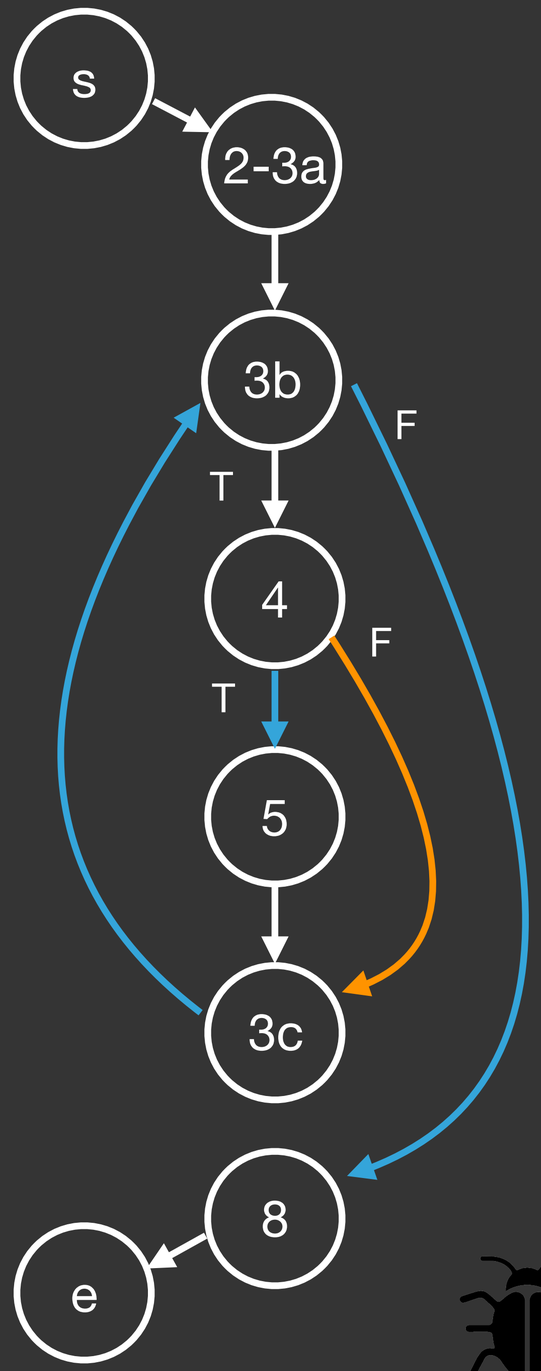
The test suite should execute all true/false edges of the CFG

The test case x = [0] and x = [1] would execute all true/false branches

x = [0] takes the true branch from node 4, x = [1] takes the false branch

(Note: the test cases could be merged into one: x = [0, 1])

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Branch Coverage

The test suite should execute all true/false edges of the CFG

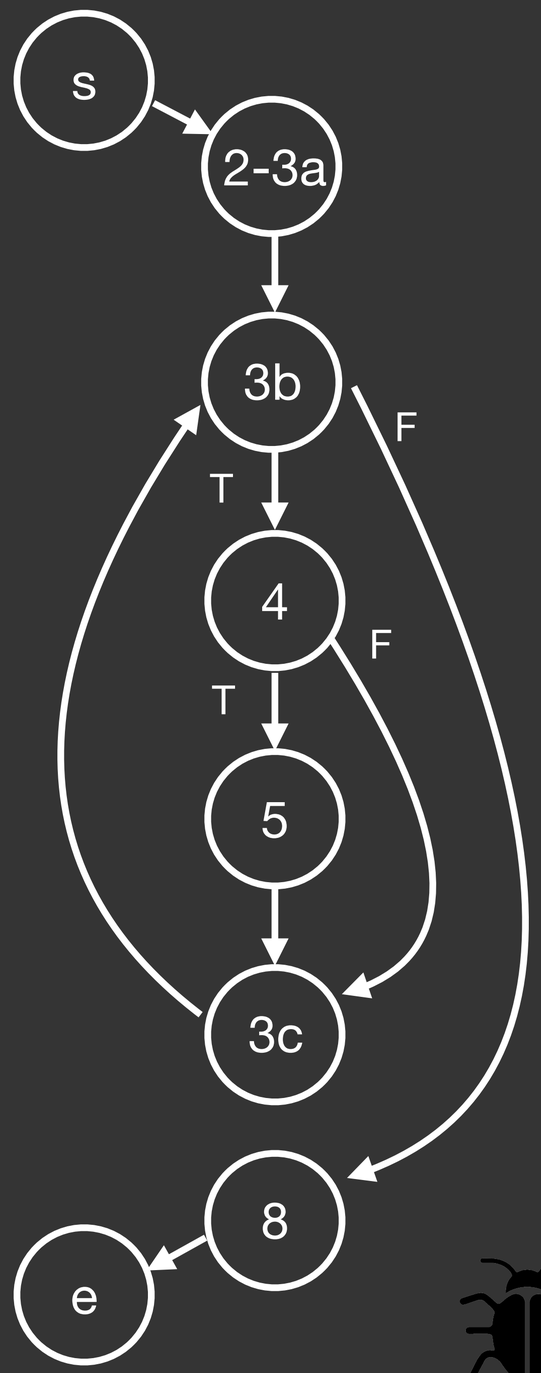
Also known as:

- Decision Coverage
- Predicate Coverage
- Edge Coverage

Path Coverage

The test suite should execute all paths through the CFG

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Path Coverage

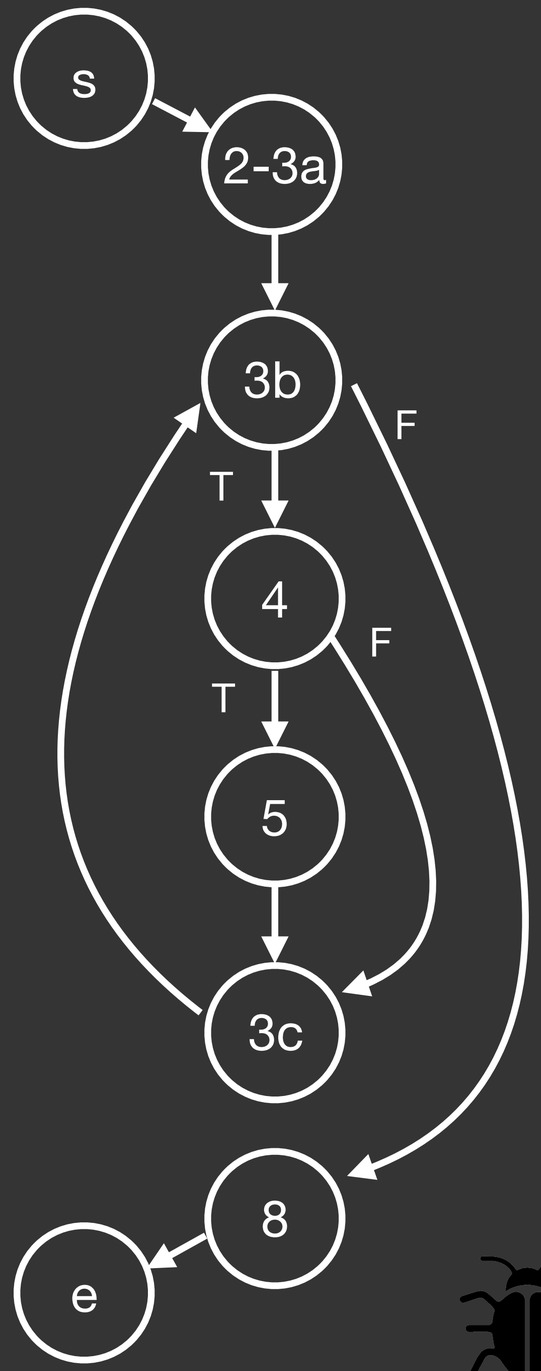
The test suite should execute all paths through the CFG

Usually not possible in practice

The number of paths through `countZeros` is dependent on the length of x

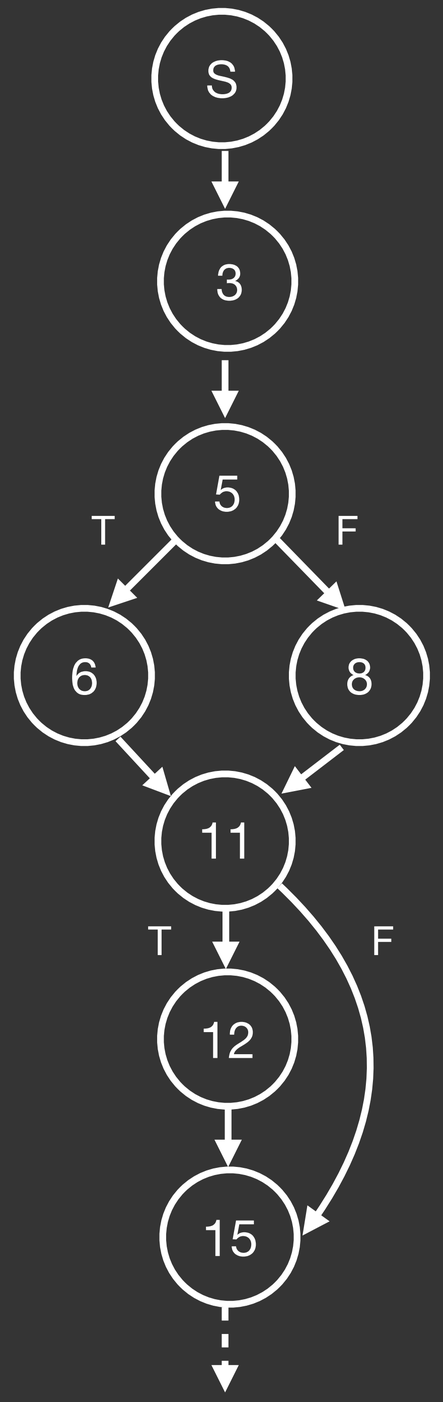
Some versions of Path Coverage concentrate on 0, 1 or more executions of every loop to mitigate potentially infinite numbers of paths

```
public int countZeros(int[] x) {
  int count = 0;
  for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
    if (x[i] == 0) {
        count ++;
    }
  }
  return count;
}
```



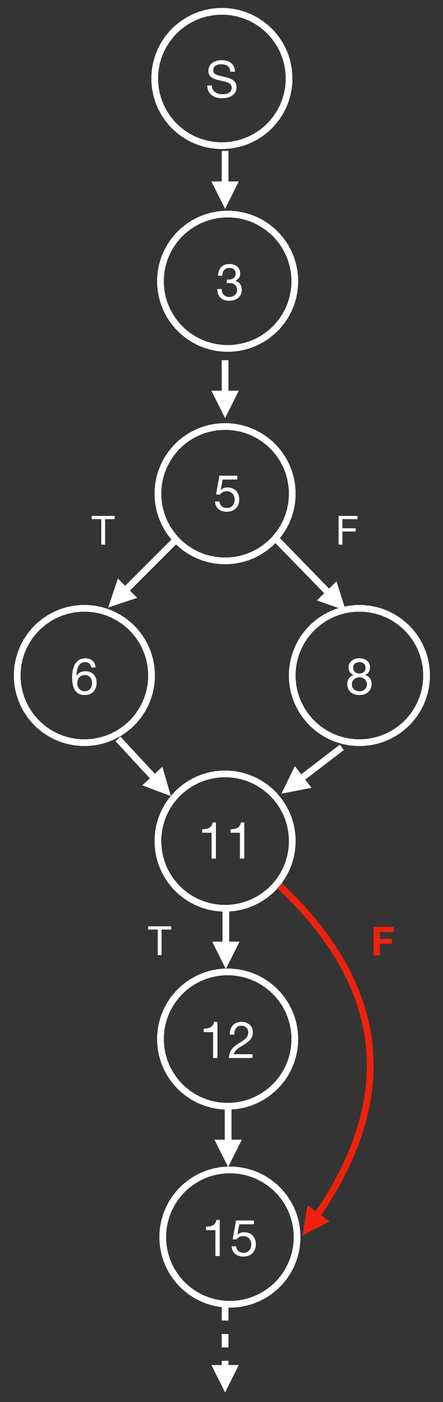
There is a problem lurking here for Branch Coverage. What is it?

```
1  public void testMe(int x) {
2
3    int y = 0;
4
5    if (x > 0) {
6        y = y + 1;
7    } else {
8        y = y + 2;
9    }
10
11   if (y > 0) {
12       y = y + 1;
13   }
14
15   // ...
```



There is a problem lurking here for Branch Coverage. What is it?

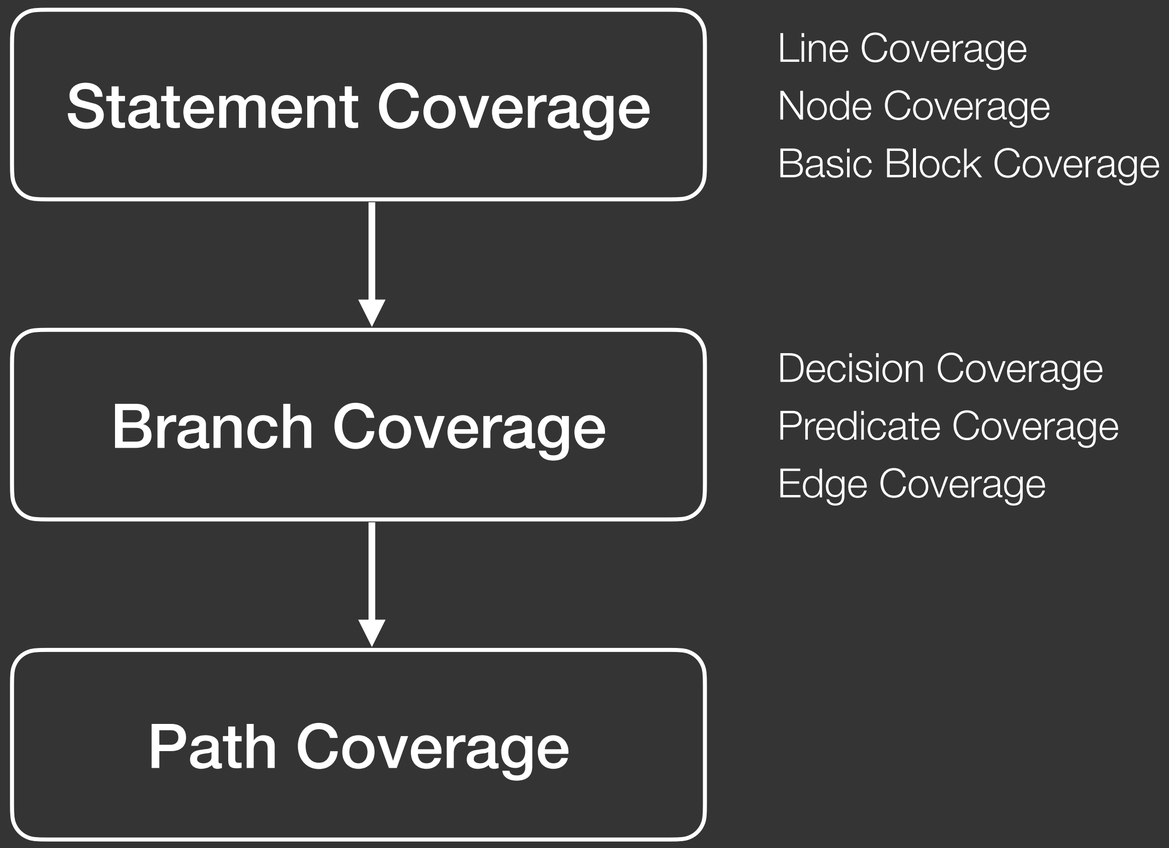
```
1  public void testMe(int x) {
2
3    int y = 0;
4
5    if (x > 0) {
6        y = y + 1;
7    } else {
8        y = y + 2;
9    }
10
11   if (y > 0) {
12       y = y + 1;
13   }
14
15   // ...
```



Coverage Criteria are Subject To Infeasible Test Requirements

- This *can* also happen with Statement Coverage
    - Infeasible statements point to dead code
- It is *more likely* to happen with Branch Coverage
    - Infeasible branches point to redundant decisions in the code
- It is *very likely* with Path Coverage
    - Infeasible paths are not necessarily the result of redundancy
    - Not all the paths through the CFG are legitimately possible in the actual code

Subsumption of Structural Coverage Criteria



Line Coverage
Node Coverage
Basic Block Coverage

Decision Coverage
Predicate Coverage
Edge Coverage

When to Use Structural Coverage

- Structural coverage level is a useful metric to understand how much of your code is executed by your test suite.
- Common Rationale: You wouldn't want to release parts of your code that weren't exercised at least once by at least one test.
  - As such Statement/Line Coverage is a commonly used metric
  - But Branch Coverage is stronger, and obtainable without much more additional effort
  - Path Coverage is less common and often intractable

White-Box Coverage Criteria based on

Logic Analysis

Is Branch Coverage Enough?

Consider the following defect in a method that checks if a user is allowed to vote in the UK (voters must be British and 18+):

```
if (nationality != "BRITISH" || age < 21)
    throw new Exception("Cannot vote”)
```

Branch coverage can be satisfied by testing only the sub-condition involving the nationality, i.e., the faulty sub-expression may never determine the outcome even if we test both outcomes of the branch.

Logic-based Coverage aims to test individual conditions in compound predicates.

```java
public static int daysBetweenTwoDates(int year1, int month1, int day1,
                                      int year2, int month2, int day2) {
  ...
  if ((year2 < year1) ||
          (year2 == year1 && month2 < month1) ||
          (year2 == year1 && month2 == month1 && day2 < day1)) {
      int t = month2;
      month2 = month1;
      month1 = t;
      t = day2;
      day2 = day1;
      day1 = t;
      t = year2;
      year2 = year1;
      year1 = t;
  }
  ...
```

$$D_1 \vee D_2$$

Disjunct 1

Disjunct 2

Disjunct 3

```
public static int daysBetweenTwoDates(int year1, int month1, int day1,
                                      int year2, int month2, int day2) {
  ...
  if ((year2 < year1) ||
        (year2 == year1 && month2 < month1) ||
        (year2 == year1 && month2 == month1 && day2 < day1)) {
      int t = month2;
      month2 = month1;
      month1 = t;
      t = day2;
      day2 = day1;
      day1 = t;
      t = year2;
      year2 = year1;
      year1 = t;
  }
  ...
```

Disjunct 1

Disjunct 2

Disjunct 3

```
public static int daysBetweenTwoDates(int year1, int month1, int day1,
                                      int year2, int month2, int day2) {
  ...
  if ((year2 < year1) ||
      (year2 == year1 && month2 < month1) ||
      (year2 == year1 && month2 == month1 && day2 < day1)) {
      int t = month2;
      month2 = month1;
      month1 = t;
      t = day2;
      day2 = day1;
      day1 = t;
      t = year2;
      year2 = year1;
      year1 = t;
  }
  ...
```

Branch Coverage does not ensure each disjunct is exercised as true and false:

Test Case	Example Input						Disjunct			Branch Predicate
	year1	month1	day1	year2	month2	day2	1	2	3	
1	2019	12	13	2018	4	25	T	F	F	T
2	2018	4	25	2019	12	13	F	F	F	F

Analysing the Logic of a Predicate

Logical Operators

Operator	Math Symbol	Java
not	¬	!
and	∧	&&
or	∨	| |

A condition is a boolean expression that is a component of a more complex predicate that does not contain any logical operators

Analysing the Logic of a Predicate

Logical Operators

Operator	Math Symbol	Java
not	¬	!
and	∧	&&
or	∨	||

A condition is a boolean expression that is a component of a more complex predicate that does not contain any logical operators

```
if ((year2 < year1) ||
        (year2 == year1 && month2 < month1) ||
        (year2 == year1 && month2 == month1 && day2 < day1)) {
  ...
}
```

How many conditions are there in this `if` statement?

Analysing the Logic of a Predicate

```
if ((year2 < year1) ||
        (year2 == year1 && month2 < month1) ||
        (year2 == year1 && month2 == month1 && day2 < day1)) {
  ...
}
```

How many conditions are there in this `if` statement?

	Condition	Disjunct
1	year2 < year1	1
2	year2 == year1	2
3	month2 < month1	2
=2	year2 == year1	3
4	month2 == month1	3
5	day2 < day1	3

Condition Coverage

Exercise each condition as **true** and **false**

```
if ((year2 < year1) ||
        (year2 == year1 && month2 < month1) ||
        (year2 == year1 && month2 == month1 && day2 < day1)) {
  ...
}
```

	Condition	Disjunct
1	year2 < year1	1
2	year2 == year1	2
3	month2 < month1	2
=2	year2 == year1	3
4	month2 == month1	3
5	day2 < day1	3

Test Case	Condition 1	2	3	4	5	Branch Predicate	Example Input year1	month1	day1	year2	month2	day2
1	T	F	F	T	T	T	2019	1	2	2018	1	1
2	F	T	T	F	F	T	2019	2	1	2019	1	1

Multiple Condition Coverage

Exercise each possible combination of truth values for each condition

n conditions = 2^n test requirements

Multiple Condition Coverage

Exercise each possible combination of truth values for each condition

n conditions = 2^n test requirements

	Condition	Disjunct
1	`year2 < year1`	1
2	`year2 == year1`	2
3	`month2 < month1`	2
=2	`year2 == year1`	3
4	`month2 == month1`	3
5	`day2 < day1`	3

Multiple Condition Coverage

Exercise each possible combination of truth values for each condition

n conditions = 2^n test requirements

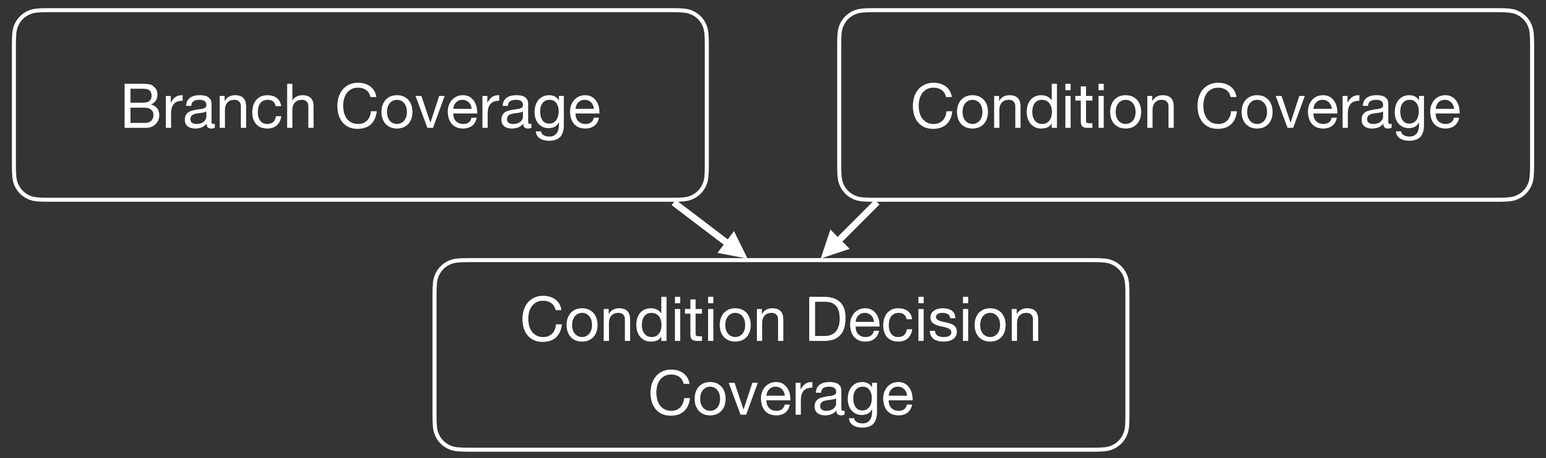
	Condition	Disjunct
1	`year2 < year1`	1
2	`year2 == year1`	2
3	`month2 < month1`	2
=2	`year2 == year1`	3
4	`month2 == month1`	3
5	`day2 < day1`	3

5 unique conditions = 2^5 = 32 test requirements

But not all combinations are **feasible**

Condition Decision Coverage

Exercise each condition as true and false, as well as the overall branch predicate



Modified Condition Decision Coverage (MCDC)

Exercise each condition as true and false. When the condition flips truth value, the branch predicate must also flip

Modified Condition Decision Coverage (MCDC)

Exercise each condition as true and false.
When the condition flips truth value, the branch predicate **must also** flip

The "major" condition

Truth values of the remaining "minor" conditions set so that flipping the major condition flips the predicate

The major condition will then determine the predicate

	Condition	Disjunct
1	`year2 < year1`	1
2	`year2 == year1`	2
3	`month2 < month1`	2
=2	`year2 == year1`	3
4	`month2 == month1`	3
5	`day2 < day1`	3

Restricted MCDC

The truth values of the minor conditions are fixed as the major condition and the predicate flip

Condition 1 is major:

Test Case	Condition 1	2	3	4	5	Branch Predicate	Example Input year1	month1	day1	year2	month2	day2
1	*T*	F	F	F	F	*T*	2019	1	1	2018	2	1
2	*F*	F	F	F	F	*F*	2018	1	1	2019	2	1

Condition 2 is major:

Test Case	Condition 1	2	3	4	5	Branch Predicate	Example Input year1	month1	day1	year2	month2	day2
3	F	*T*	T	F	F	*T*	2019	2	1	2019	1	1
4	F	*F*	T	F	F	*F*	2018	2	1	2019	1	1

	Condition	Disjunct
1	`year2 < year1`	1
2	`year2 == year1`	2
3	`month2 < month1`	2
=2	`year2 == year1`	3
4	`month2 == month1`	3
5	`day2 < day1`	3

Correlated MCDC

The truth values of the minor conditions do not need to be fixed as the major condition and the predicate flip

Test Case	Condition 1	2	3	4	5	Branch Predicate	Example Input year1	month1	day1	year2	month2	day2
1	T	F	F	T	T	T	2019	1	2	2018	1	1
2	F	T	T	F	F	T	2019	2	1	2019	1	1
3	F	F	F	F	F	F	2018	1	1	2019	2	1

Correlated MCDC

The truth values of the minor conditions do not need to be fixed as the major condition and the predicate flip

Test Case	Condition 1	2	3	4	5	Branch Predicate	Example Input year1	month1	day1	year2	month2	day2
1	T	F	F	T	T	T	2019	1	2	2018	1	1
2	F	T	T	F	F	T	2019	2	1	2019	1	1
3	F	F	F	F	F	F	2018	1	1	2019	2	1

Correlated MCDC

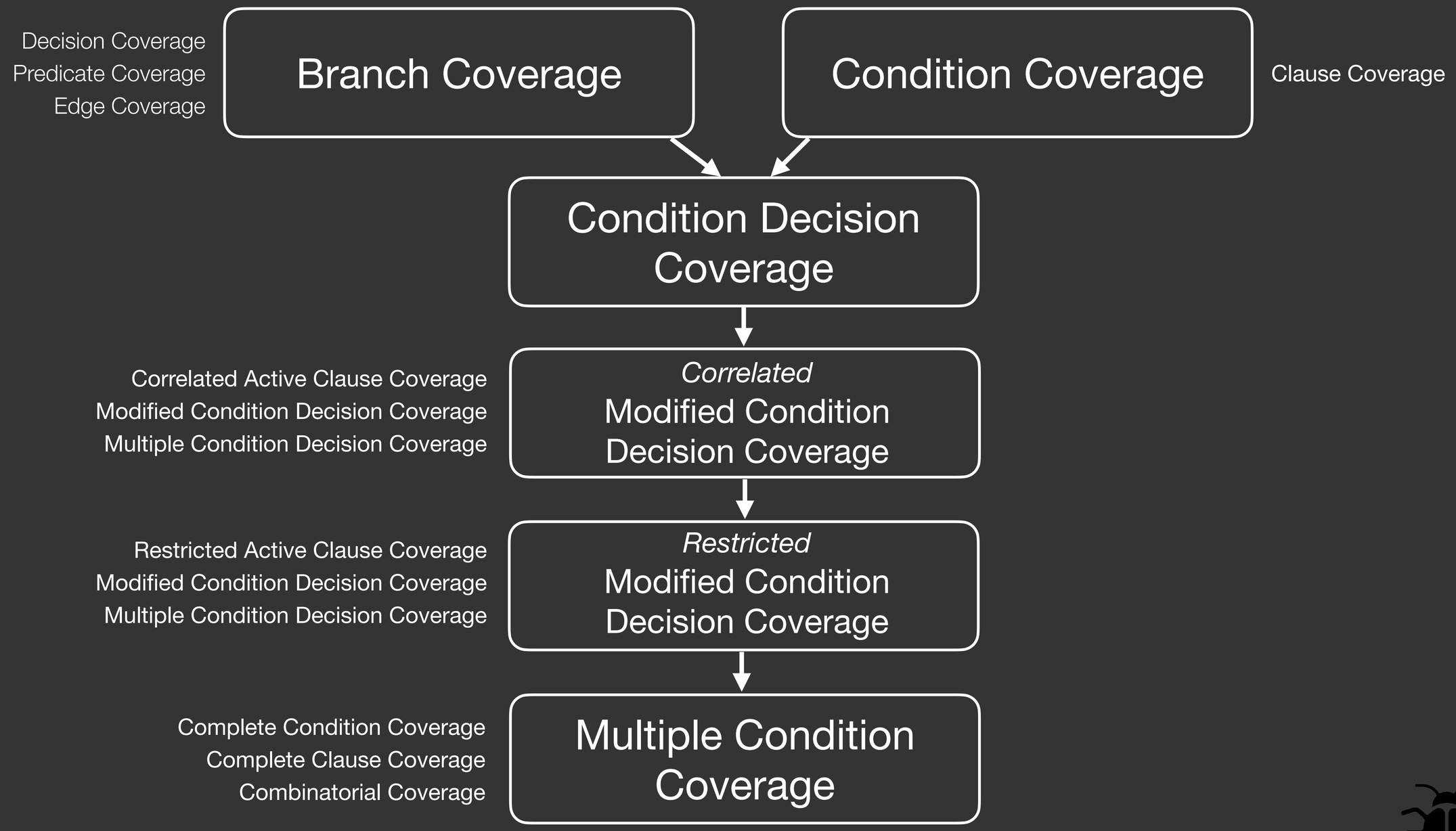
The truth values of the minor conditions do not need to be fixed as the major condition and the predicate flip

Test Case	Condition 1	2	3	4	5	Branch Predicate	Example Input year1	month1	day1	year2	month2	day2
1	T	F	F	T	T	T	2019	1	2	2018	1	1
2	F	T	T	F	F	T	2019	2	1	2019	1	1
3	F	F	F	F	F	F	2018	1	1	2019	2	1

	Condition	Disjunct
1	`year2 < year1`	1
2	`year2 == year1`	2
3	`month2 < month1`	2
=2	`year2 == year1`	3
4	`month2 == month1`	3
5	`day2 < day1`	3

```
if ((year2 < year1) ||
        (year2 == year1 && month2 < month1) ||
        (year2 == year1 && month2 == month1 && day2 < day1)) {
  ...
}
```

Disjunct 3 not actually exercised as true


Decision Coverage
Predicate Coverage
Edge Coverage
Branch Coverage
Condition Coverage
Clause Coverage
Condition Decision Coverage
Correlated Active Clause Coverage
Modified Condition Decision Coverage
Multiple Condition Decision Coverage
Correlated
Modified Condition Decision Coverage
Restricted Active Clause Coverage
Modified Condition Decision Coverage
Multiple Condition Decision Coverage
Restricted
Modified Condition Decision Coverage
Complete Condition Coverage
Complete Clause Coverage
Combinatorial Coverage
Multiple Condition Coverage

Some General Observations

- MCDC variants offer the best trade-off in terms of number of test requirements v. test thoroughness
- Correlated MCDC tends to result in smaller test suites than Restricted
- Multiple Condition Coverage yields too many test requirements
- Condition Coverage does not subsume Branch Coverage

Consider this…

```
...
boolean yearSame = year2 == year1;
boolean yearAndMonthSame = yearSame && month2 == month1;

boolean secondDateBeforeFirstByYear = year2 < year1;
boolean secondDateBeforeFirstByMonth = yearSame && month2 < month1;
boolean secondDateBeforeFirstByDay = yearAndMonthSame && day2 < day1;

boolean secondDateBefore = secondDateBeforeFirstByYear ||
            secondDateBeforeFirstByMonth || secondDateBeforeFirstByDay;

if (secondDateBeforeFirst) {
    ...
}
...
```

MCDC and Testing Standards

- ISO 26262 – “Road Vehicles – Functional Safety”
  https://en.wikipedia.org/wiki/ISO_26262

- EN 50128 – a functional safety standard used in the rail industry
  https://www.adacore.com/industries/rail/en50128

- DO-178B and DO-178C – “Software Considerations in Airborne Systems and Equipment Certification”
  https://en.wikipedia.org/wiki/DO-178B
  https://en.wikipedia.org/wiki/DO-178C

- IEC 61508, a basic functional safety standard applicable to all industries.
  https://en.wikipedia.org/wiki/IEC_61508

University of Sheffield

COM3529 Software Testing and Analysis

Coverage Criteria: Conclusions

Dr José Miguel Rojas

The Pros and Cons of Coverage

- The **“Cons”**
  - They can generate **too many** test requirements
  - Not all test requirements are **useful**
  - Higher coverage doesn’t guarantee that more bugs will be found
- The **“Pros”**
  - Coverage criteria ensure that our test suites have certain properties or strengths
  - Coverage criteria tell us **how much** testing has been done and **when to stop testing**

Software Testing Research and Coverage Criteria

- The relationship between actual faults, failures and types of coverage and coverage levels
- Where to concentrate testing in order to find faults and failures
- What the ideal test suite size is
- How to check that a fault has already been repaired, and has not introduced further faults
- Can a fault be automatically repaired?
- …